Intro

Period 6

Name: Prince Zheng

Project Title: Xiangqi

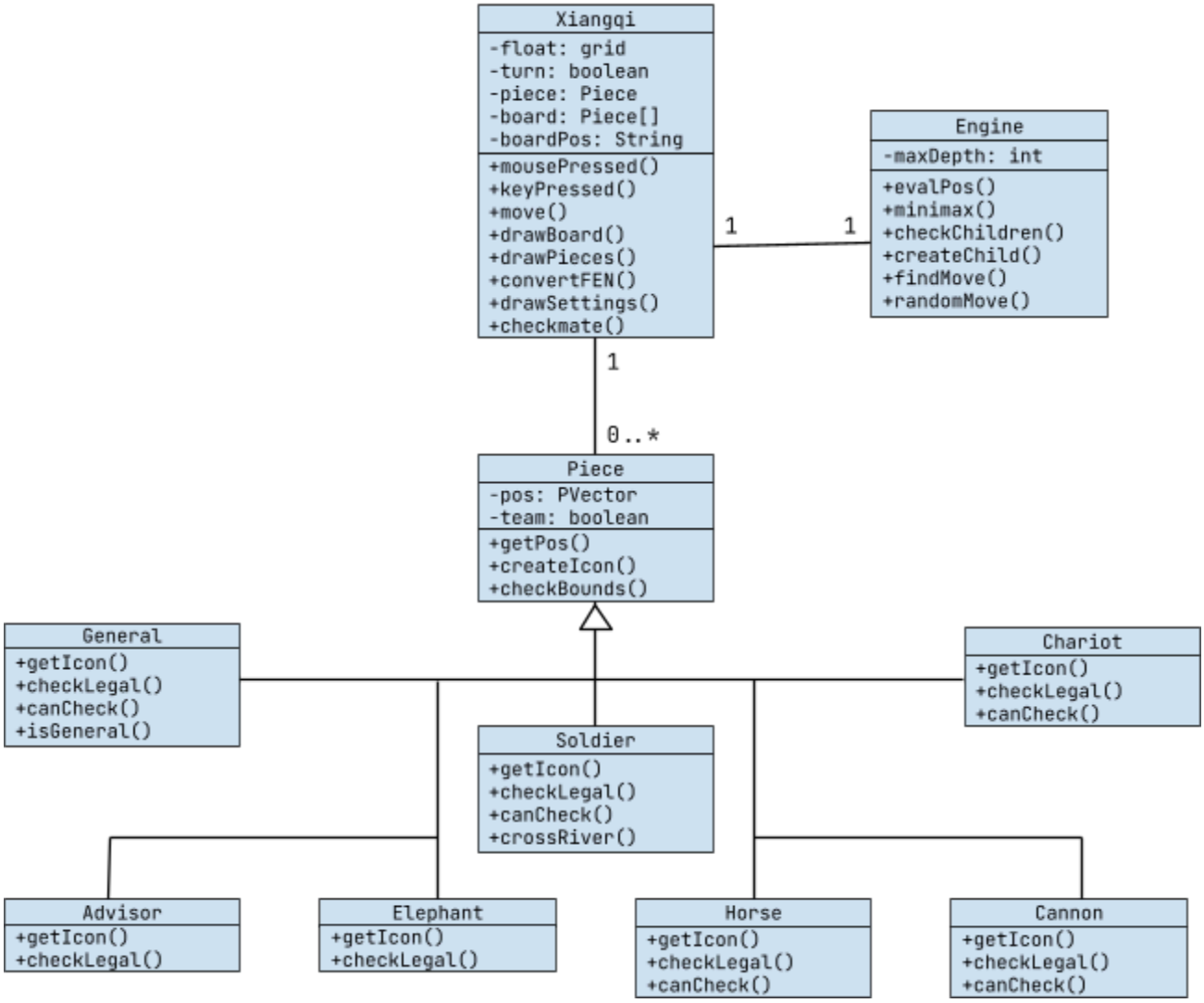
Description

The project will be a recreation of Xiangqi, or Chinese chess, a 2 player strategy board game with perfect information. It will allow 2 players to play against each other locally, or allow 1 player to play against a computer engine.

I will be using an array to store and change the board itself, but I will use Forsyth-Edwards Notation to store each position, most notably the starting position, in strings, which can be converted into an array. Each piece can be stored as an integer, and different rules will be implemented and checked based on these pieces that will determine legal moves, which are stored as move notation.

To play against a computer engine, I will be using the minimax search algorithm to “look ahead” in order to determine how good a move is for the future, using a static evaluation function to evaluate any one position in the form of an integer. Alpha-beta pruning, which removes branches that are guaranteed to be suboptimal, will also be necessary for performance, as Xiangqi has a game-tree complexity of about 10^{150}; for reference, Western chess only has about 10^{120}.

UML Diagram



How Does It Work

The objective of the game is to checkmate your opponent's general with your pieces, where the opponent will have no more legal moves. Each player has 16 pieces, labelled with various Chinese characters. Below are the complete rules of Xiangqi:

https://www.xiangqi.com/how-to-play-xiangqi

Each player takes turns moving pieces along the intersections of the board lines. The player can click on a piece and all legal positions that the piece can move to will be highlighted as dots on the board. Clicking one of these dots will move the piece to that position or capture a piece on that position. Clicking anywhere else will allow the player to select a different piece. The player will then have to wait for the other player or the computer engine to make their move before they can move again.

Functionalities / Issues

- Board and Piece Rendering: drawBoard()
    - Grid of 9 files x 10 ranks
    - Draws grid lines, the river, borders, piece position markings, palace lines.
    - drawPieces()
        - Displays an image of each piece on the board
- Board Initialization: convertFEN()
    - Initializes the board by parsing the boardPos FEN
    - Creates Piece objects and adds them to the board array
        - Piece subclasses: Advisor, Cannon, Chariot, Elephant, General, Horse, Soldier
    - Declares turn with ‘r’ or ‘b’
- Piece Selection
    - Determines what piece is selected based on where the user clicks
    - A circle is created to highlight selected pieces

        - *Issue of highlights would remain after clicking something else: restructured the code so the board could be reset seamlessly*
    - Deselect piece when move is not made
    - Shows all of the legal moves with small transparent circles
- Piece Movement
    - If a piece is selected, checks if where the user clicks is move is one of the piece's legal moves and if so the piece is moved to that position
        - *Issue of constantly resetting board position, restructured variables*
    - Switches turns after moving
    - Prevents moves entering check
        - *Issue of Stack Overflow errors when checking future board state for discovery checks: created simpler method for each piece for checks*
- Checkmate Detection
    - Shows the winner when a player has no more legal moves
    - Resets the game board after small delay
- Settings Menu: drawSettings()
    - Shows upon opening the game, press TAB to exit
        - Reset the game and reopen settings by pressing TAB
    - Switch piece styles, scale the board size (window size), switch game modes, switch player teams, and toggle sound effects using buttons
        - Switching teams only available when playing against a bot, implemented flipping the board and randomization
    - Open website with rules for pieces and moves by pressing ENTER

- Engine Movement
    - Evaluates any static position as a float
    - Minimax with Alpha-beta pruning
        - Explores all possible outcomes to a certain depth recursively: minimizes the maximum possible loss after evaluating each position
        - Optimize by removing branches that don't need to be explored because there is already a better known value
    - Takes any board position in the form of an array and create children board positions based on possible moves, used for exploring
    - FindMove()
        - Returns next move based on best minimax result
        - *Issues with optimization: limited amount of depth because of very high time complexity*
    - RandomMove()
        - Returns any legal move at random, randomly called to help change up the pacing of the game!